Public - عام

ARTI 502

Deep Learning: 1st Assignment

Training-Testing Neural Net with MNIST Dataset using Pytorch

Year 2026 Term 1

Group #

Group Members:

#	Member Name	Member Academic ID
1	Fatimah Hassan Sowileh	2220006563
2	Batool Ayman Aldauwagh	2230003843
3	Najaf Alsabia	2230004326
4	Fadak Mohammed Alqassab	2230006504
5	Hawra Jafar Al-Yosuif	2230004521
6	Kawthar Hussain Althunayan	2230001860

Overview

In this assignment, you will perform Training Neural Net on the MNIST Dataset dataset using Pytorch. The goal is to explore the Basic Neural Network, understand its structure, and identify important issues such as architecture, learning rate, momentum and other parameters. You will also analyze relationships between number of layers and number of neurons for each layers to the overall Neural Network performance through their tuning hyperparameters.

Public - عام

Public - عام

Objective

Explore the dataset using PyTorch to understand:

- Simple Neural Networks structure
- How to do Training-Testing Neural Networks
- How to evaluate the performance
- How to determine learning rate, momentum and model hyperparameters

Tasks (with PyTorch)

Task 1: Install PyTorch

Instructions:

- Install PyTorch and load it to your python code and save it as **'Assigment-1 DL'**
- Screenshoot how your load PyTorch to your python source code.

```
# Task 1: Import and verify PyTorch

import torch

# Display the installed PyTorch version
print("PyTorch version:", torch.__version__)

# Confirm successful import
print("PyTorch loaded successfully!")
```

```
PyTorch version: 2.11.0+cpu
PyTorch loaded successfully!
```

Task 2: Download and Load the Dataset

Instructions:

- PyTorch has some built tools for downloading and loading common datasets. You will be playing around with MNIST in this assignment. It is a dataset of 28x28 grayscale handwritten digits 0-9. There are 50,000 images in the training set and 10,000 in the test set.

Public - عام

Public - عام

- Download the dataset directly from your python code then load it into the the workspace.

Deliverable:

- Screenshot showing the dataset loaded successfully.

```
Training samples: 50000
Testing samples: 10000
MNIST dataset loaded successfully!
```

Task 3: Inspect the Dataset

Instructions:

- It's worthwhile to check out how the dataloader loads the images into tensors. You can print out the size of the loaded data in the images tensors. Also you can use matplotlib to see the visualization of the images data

Deliverable:

- Provide screenshots how to find:
  1. Images tensor size

```python
# Task 3: Inspect the Dataset

# Get one batch of images and labels
images, labels = next(iter(train_loader))

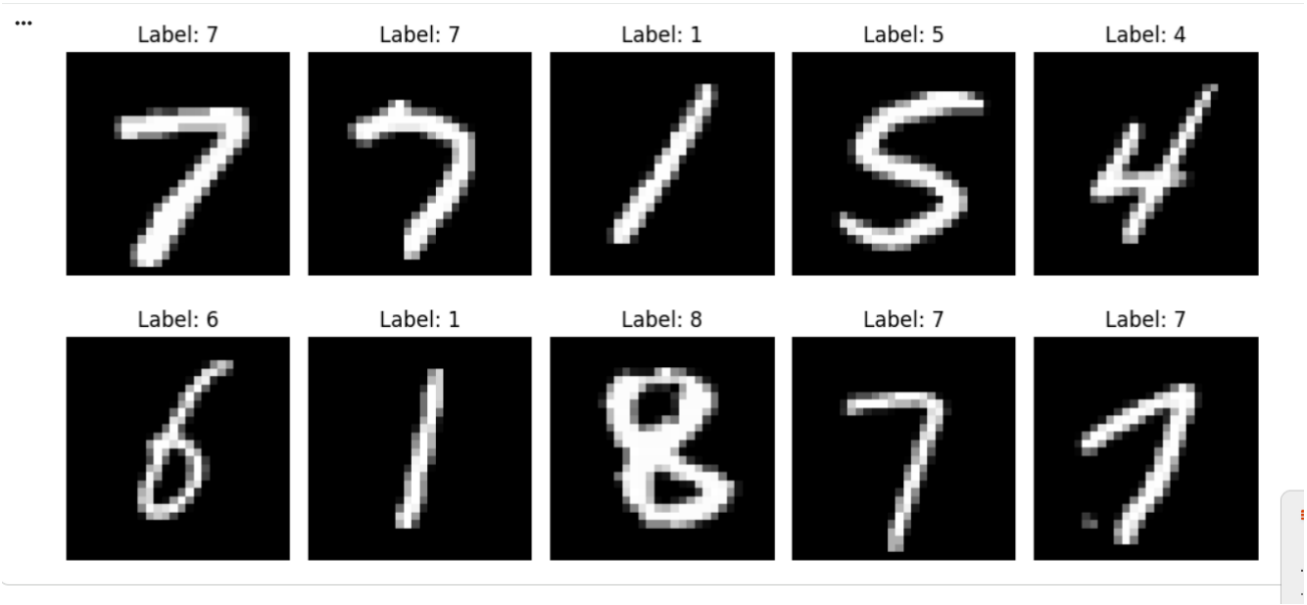
# Display the tensor dimensions
print("Images tensor size:", images.shape)
print("Labels tensor size:", labels.shape)
```

```
Images tensor size: torch.Size([32, 1, 28, 28])
Labels tensor size: torch.Size([32])
```

Public - عام

Public - عام

2. Data images vizualization using matplotlib



Task 4: Creating A Simple Network

Instructions:

- Write your Python code to setup the following simple network architecture:

  First you 'll build a very basic neural network with a single hidden layer of neurons. This means one fully connected layer of weights connects the input to the hidden neurons and one fully connected layer connects the hidden neurons to the output. Use the RELU activation function on the hidden neuron values as our nonlinearity.

  These fully connected (or nn.Linear) layers expect a 2D input tensor where that is N x I where N is the number of data points in a **mini batch** and I is the **number of inputs**. However, our data is formatted in (N x C x H x W) right now so we need to tell PyTorch to rearrange it using torch.flatten. C, H and W are refer to Color, Height and Width respectively.

Public - عام

Public - عام

After being **flattened** our data goes from (32 x 1 x 28 x 28) to being (32 x 784)

Deliverables:

- Screenshot of source code

```python
# Task 4 : Creating A Simple Network
import torch.nn as nn
import torch.nn.functional as F

class SimpleNet(nn.Module):
    def __init__(self, input_size=784, hidden_size=128, num_classes=10):
        super(SimpleNet, self).__init__()
        # First fully connected layer: connects input to hidden neurons
        self.fc1 = nn.Linear(input_size, hidden_size)
        # Second fully connected layer: connects hidden neurons to output classes
        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        # Flatten the input tensor from (N, C, H, W) to (N, I) -> (32, 1, 28, 28) to (32, 784)
        x = torch.flatten(x, start_dim=1)


        # Hidden layer with ReLU activation
        x = F.relu(self.fc1(x))

        # Output layer
        x = self.fc2(x)
        return x

# Instantiate the model
model = SimpleNet()
print(model)

# --- Verification Step ---
# Pass one batch from Task 3 to verify tensor dimensions and forward pass
test_batch_images, test_batch_labels = next(iter(train_loader))
```

```python
print("\n--- Verification ---")
print("Input batch shape :", test_batch_images.shape)
print("Output batch shape:", output.shape)
print("Expected output   : torch.Size([32, 10])")
```

- Verify your code with above requirement.

```
SimpleNet(
  (fc1): Linear(in_features=784, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)

--- Verification ---
Input batch shape : torch.Size([32, 1, 28, 28])
Output batch shape: torch.Size([32, 10])
Expected output   : torch.Size([32, 10])
```

Task 5: Training Function

Instructions:

- Now for training the network. Your train function takes as input the net to train and the dataloader for the training data. It also takes some optional parameters to control training.

Public - عام

Public - عام

- For your network you will be using PyTorch's built in nn.CrossEntropyLoss. This will apply a softmax to our network's output, calculate the log-probability assigned to each class, then try to minimize the negative log likelihood of our data (AKA maximize the likelihood)
- For your optimizer you are using stochastic gradient descent with learning rate and momentum

Deliverables:

- Screenshot of your python source code that contains your training function with above requirements



```python
import torch
import torch.nn as nn
import torch.optim as optim

def train(net, train_loader, epochs=5, learning_rate=0.01, momentum=0.9):
    # Define the loss function
    criterion = nn.CrossEntropyLoss()

    # Define the optimizer
    optimizer = optim.SGD(
        net.parameters(),
        lr=learning_rate,
        momentum=momentum
    )

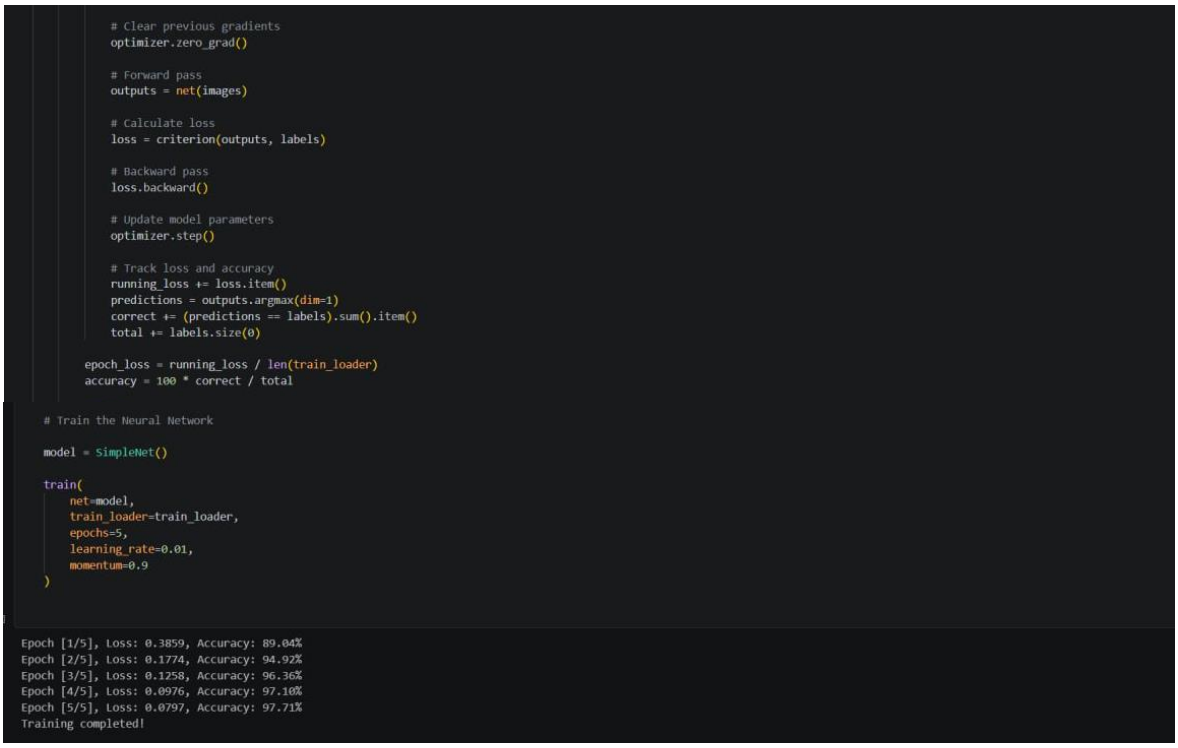
    # Train the network
    net.train()

    for epoch in range(epochs):
        running_loss = 0.0
        correct = 0
        total = 0

        for images, labels in train_loader:
            # Flatten images from (batch_size, 1, 28, 28) to (batch_size, 784)
            images = images.view(images.size(0), -1)

            # Clear previous gradients
            optimizer.zero_grad()

            # Forward pass
```



```python
            # Clear previous gradients
            optimizer.zero_grad()

            # Forward pass
            outputs = net(images)

            # Calculate loss
            loss = criterion(outputs, labels)

            # Backward pass
            loss.backward()

            # Update model parameters
            optimizer.step()

            # Track loss and accuracy
            running_loss += loss.item()
            predictions = outputs.argmax(dim=1)
            correct += (predictions == labels).sum().item()
            total += labels.size(0)

        epoch_loss = running_loss / len(train_loader)
        accuracy = 100 * correct / total

# Train the Neural Network

model = SimpleNet()

train(
    net=model,
    train_loader=train_loader,
    epochs=5,
    learning_rate=0.01,
    momentum=0.9
)
```

```
Epoch [1/5], Loss: 0.3859, Accuracy: 89.04%
Epoch [2/5], Loss: 0.1774, Accuracy: 94.92%
Epoch [3/5], Loss: 0.1258, Accuracy: 96.36%
Epoch [4/5], Loss: 0.0976, Accuracy: 97.10%
Epoch [5/5], Loss: 0.0797, Accuracy: 97.71%
Training completed!
```

Public - عام

Public - عام

Task 6: Training Network

Instructions:

- You will instantiate a new network and train it on your training data.
- Your training function prints out some debug information about the epoch, batch number, and current loss and accuracy values. It also returns a list of all the losses on your mini-batches so you can plot them all once training has finished.

Deliverables:

- Screenshot of instantiate training source code

```
# Task 6: Instantiate a NEW network and train it

torch.manual_seed(42)        # for reproducible results
net = SimpleNet()            # new, untrained network from Task 4
print(net)

batch_losses, epoch_losses, epoch_accuracies = train_with_debug(
    net=net,
    train_loader=train_loader,
    epochs=5,
    learning_rate=0.01,
    momentum=0.9,
    print_every=200
)

print("Number of mini-batch losses recorded:", len(batch_losses))

```

- Screenshot of training debug which include epoch, batch number and current loss, accuracy values and graph for overall processes (loss and accuracy for each epoch).

Public - عام

Public - عام


```
SimpleNet(
  (fc1): Linear(in_features=784, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)
Epoch [1/5] | Batch [200/1563] | Current Loss: 0.4467 | Accuracy so far: 73.80%
Epoch [1/5] | Batch [400/1563] | Current Loss: 0.1872 | Accuracy so far: 81.08%
Epoch [1/5] | Batch [600/1563] | Current Loss: 0.5661 | Accuracy so far: 84.28%
Epoch [1/5] | Batch [800/1563] | Current Loss: 0.3554 | Accuracy so far: 86.00%
Epoch [1/5] | Batch [1000/1563] | Current Loss: 0.1962 | Accuracy so far: 87.00%
Epoch [1/5] | Batch [1200/1563] | Current Loss: 0.1658 | Accuracy so far: 87.85%
Epoch [1/5] | Batch [1400/1563] | Current Loss: 0.1651 | Accuracy so far: 88.55%
Epoch [1/5] | Batch [1563/1563] | Current Loss: 0.0319 | Accuracy so far: 88.95%
==> End of Epoch 1: Avg Loss = 0.3911, Accuracy = 88.95%

Epoch [2/5] | Batch [200/1563] | Current Loss: 0.3768 | Accuracy so far: 94.52%
Epoch [2/5] | Batch [400/1563] | Current Loss: 0.0978 | Accuracy so far: 94.36%
Epoch [2/5] | Batch [600/1563] | Current Loss: 0.1722 | Accuracy so far: 94.43%
Epoch [2/5] | Batch [800/1563] | Current Loss: 0.1032 | Accuracy so far: 94.54%
Epoch [2/5] | Batch [1000/1563] | Current Loss: 0.2468 | Accuracy so far: 94.53%
Epoch [2/5] | Batch [1200/1563] | Current Loss: 0.1132 | Accuracy so far: 94.59%
Epoch [2/5] | Batch [1400/1563] | Current Loss: 0.1922 | Accuracy so far: 94.68%
Epoch [2/5] | Batch [1563/1563] | Current Loss: 0.1319 | Accuracy so far: 94.76%
==> End of Epoch 2: Avg Loss = 0.1827, Accuracy = 94.76%

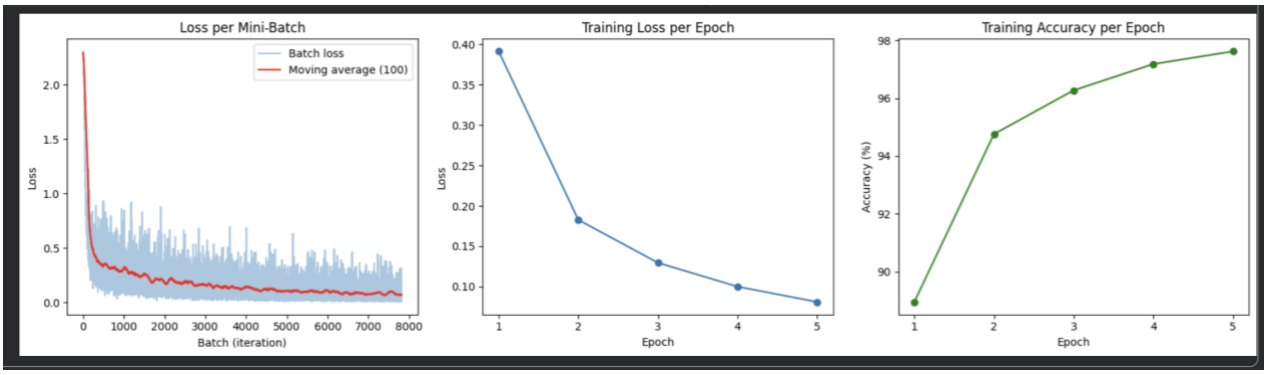
Epoch [3/5] | Batch [200/1563] | Current Loss: 0.1322 | Accuracy so far: 95.70%
Epoch [3/5] | Batch [400/1563] | Current Loss: 0.0299 | Accuracy so far: 95.94%
Epoch [3/5] | Batch [600/1563] | Current Loss: 0.0503 | Accuracy so far: 96.03%
Epoch [3/5] | Batch [800/1563] | Current Loss: 0.2307 | Accuracy so far: 96.10%
Epoch [3/5] | Batch [1000/1563] | Current Loss: 0.3879 | Accuracy so far: 96.12%
Epoch [3/5] | Batch [1200/1563] | Current Loss: 0.2659 | Accuracy so far: 96.23%
Epoch [3/5] | Batch [1400/1563] | Current Loss: 0.0913 | Accuracy so far: 96.26%
Epoch [3/5] | Batch [1563/1563] | Current Loss: 0.0532 | Accuracy so far: 96.27%
==> End of Epoch 3: Avg Loss = 0.1295, Accuracy = 96.27%
```

```
...  ==> End of Epoch 3: Avg Loss = 0.1295, Accuracy = 96.27%

Epoch [4/5] | Batch [200/1563] | Current Loss: 0.0429 | Accuracy so far: 97.08%
Epoch [4/5] | Batch [400/1563] | Current Loss: 0.1400 | Accuracy so far: 97.02%
Epoch [4/5] | Batch [600/1563] | Current Loss: 0.0533 | Accuracy so far: 97.02%
Epoch [4/5] | Batch [800/1563] | Current Loss: 0.0598 | Accuracy so far: 97.06%
Epoch [4/5] | Batch [1000/1563] | Current Loss: 0.0369 | Accuracy so far: 97.09%
Epoch [4/5] | Batch [1200/1563] | Current Loss: 0.0674 | Accuracy so far: 97.15%
Epoch [4/5] | Batch [1400/1563] | Current Loss: 0.0667 | Accuracy so far: 97.16%
Epoch [4/5] | Batch [1563/1563] | Current Loss: 0.0207 | Accuracy so far: 97.18%
==> End of Epoch 4: Avg Loss = 0.1001, Accuracy = 97.18%

Epoch [5/5] | Batch [200/1563] | Current Loss: 0.0279 | Accuracy so far: 97.80%
Epoch [5/5] | Batch [400/1563] | Current Loss: 0.0554 | Accuracy so far: 97.77%
Epoch [5/5] | Batch [600/1563] | Current Loss: 0.0489 | Accuracy so far: 97.60%
Epoch [5/5] | Batch [800/1563] | Current Loss: 0.0720 | Accuracy so far: 97.73%
Epoch [5/5] | Batch [1000/1563] | Current Loss: 0.0658 | Accuracy so far: 97.67%
Epoch [5/5] | Batch [1200/1563] | Current Loss: 0.0492 | Accuracy so far: 97.68%
Epoch [5/5] | Batch [1400/1563] | Current Loss: 0.2175 | Accuracy so far: 97.60%
Epoch [5/5] | Batch [1563/1563] | Current Loss: 0.0132 | Accuracy so far: 97.62%
==> End of Epoch 5: Avg Loss = 0.0811, Accuracy = 97.62%

Training completed!
Number of mini-batch losses recorded: 7815
```



Public - عام

Public - عام

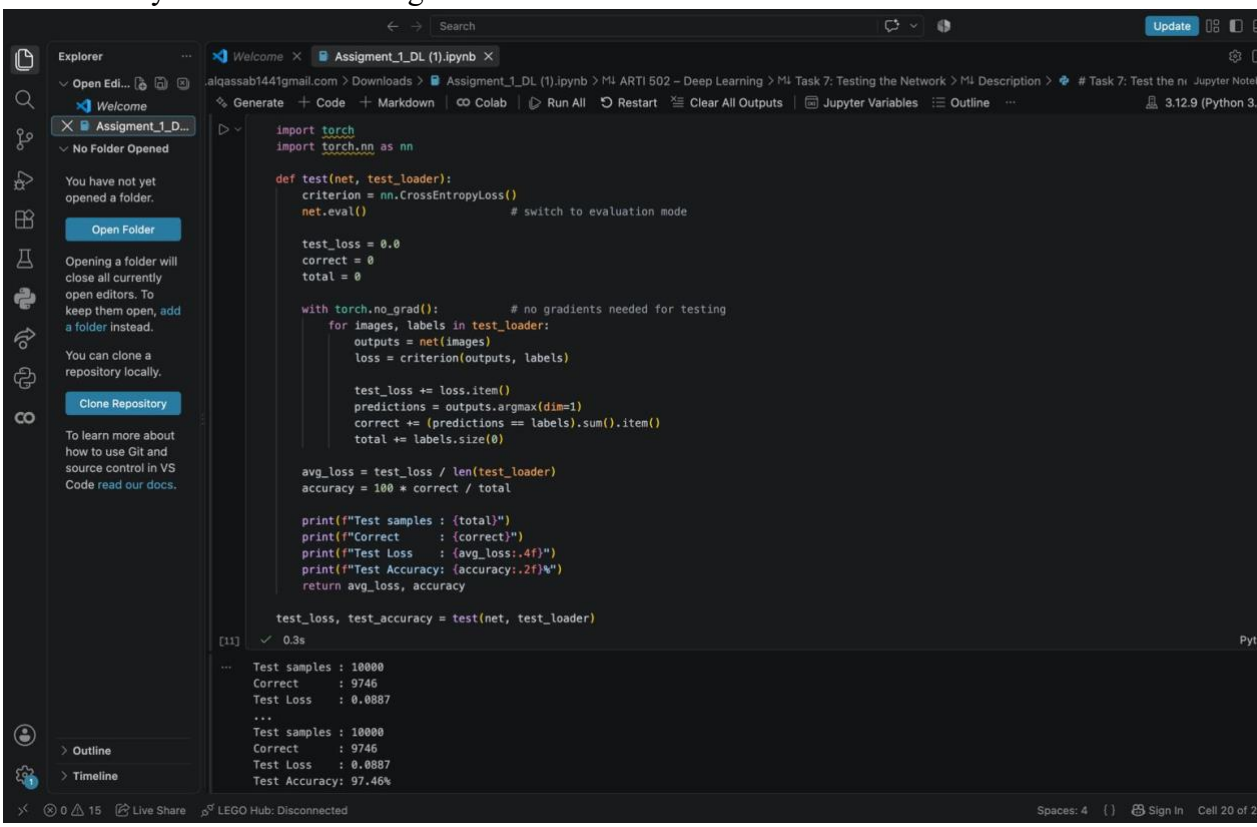
Task 7: Testing Network

Instructions:

- You can test your network on a few of your test images from task 2 and see what happens.

Deliverables:

- Screenshot your network testing source code



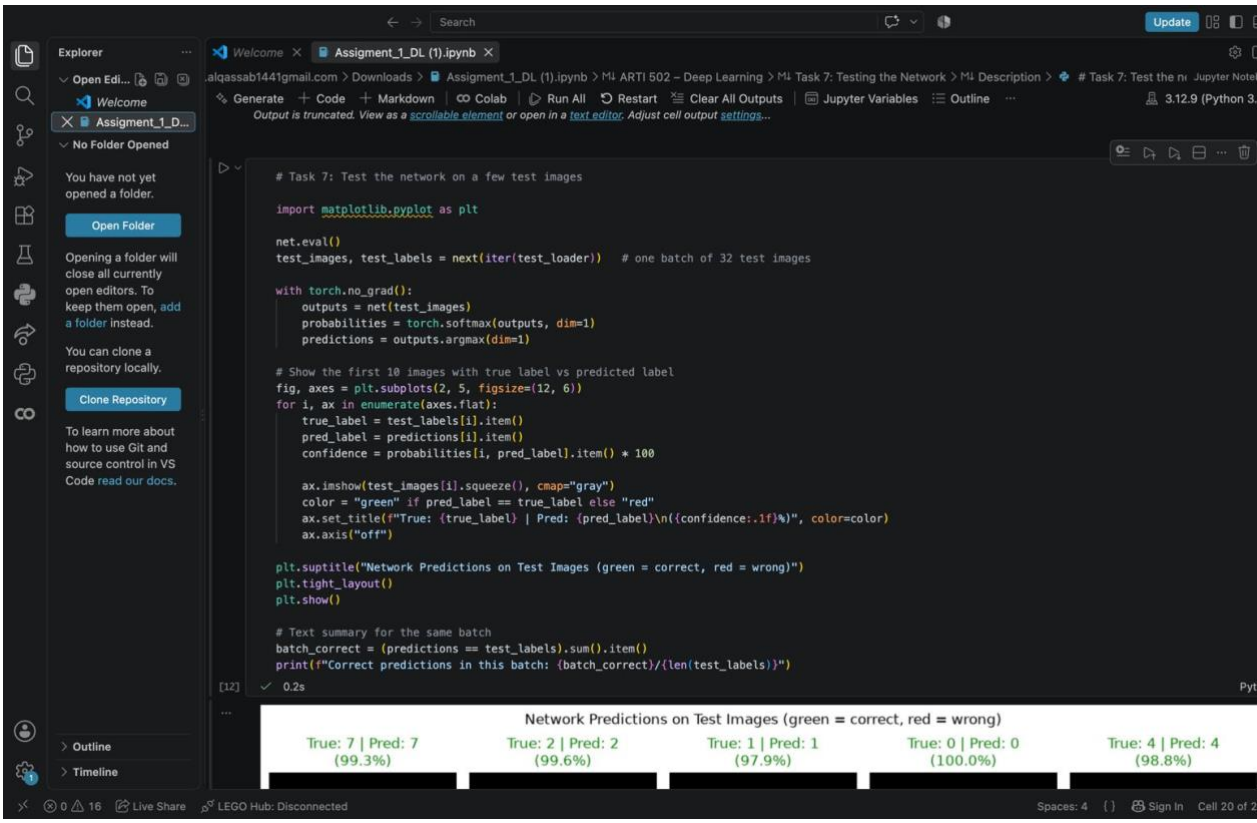
- Screenshot the results of the network testing

Public - عام

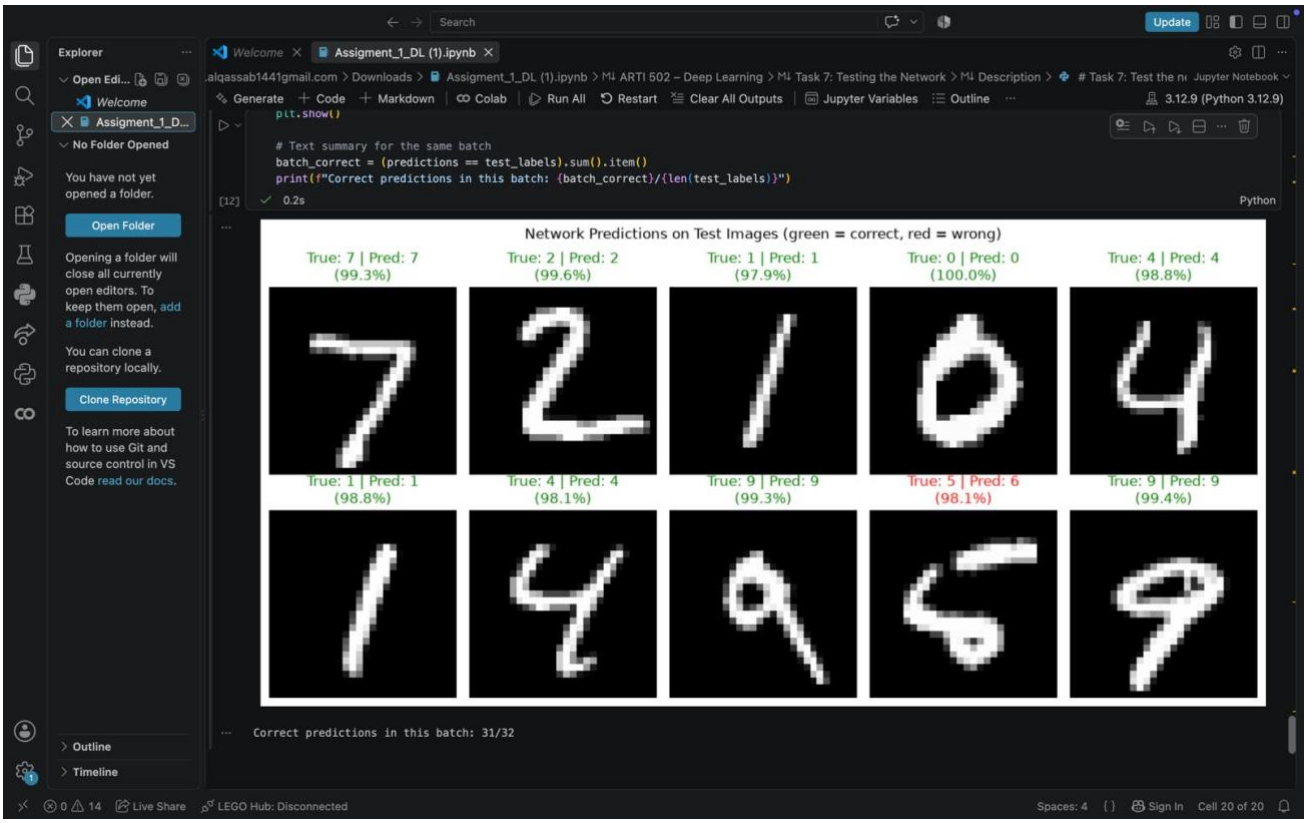
MINISTRY OF EDUCATION
IMAM ABDULRAHMAN BIN
FAISAL UNIVERSITY
COLLEGE OF COMPUTER SCIENCE
& INFORMATION TECHNOLOGY

وزارة التعليم
جامعة الإمام
عبد الرحمن بن فيصل
كلية علوم الحاسب
وتقنية المعلومات

جامعة الإمام عبد الرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY



Public - عام



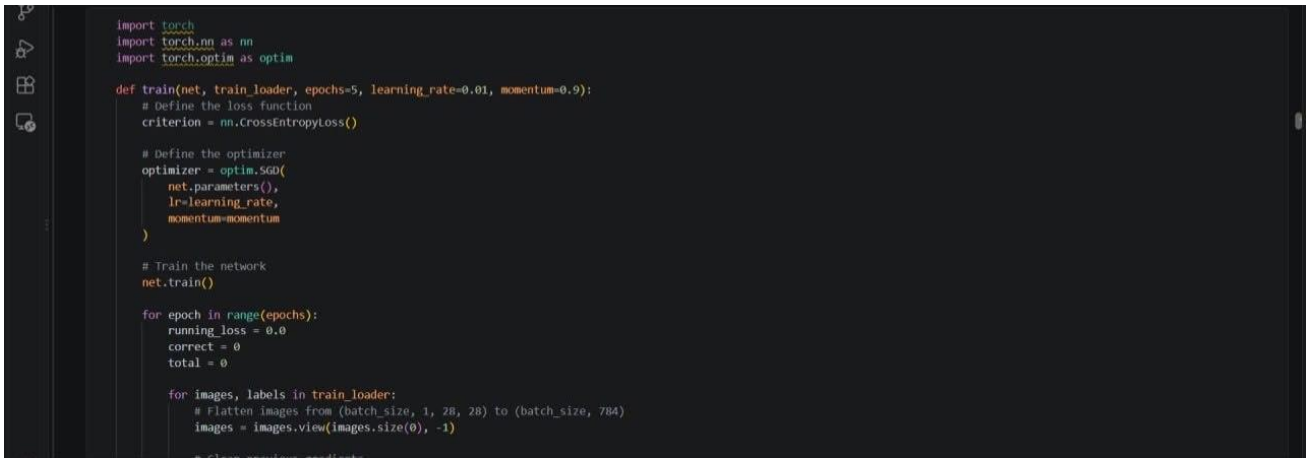
Task 8: Tuning the parameters and model archirecture

Instructions:

- Tune the parameters with different values of learning rate and momentum and write your best results (lowest loss and highest accuracy).
- Tune the architecture model with different values of number of layer and number of neuron and write your best results (lowest loss and highest accuracy).

Deliverables:

- Screen your source python code with the best results.



```
import torch
import torch.nn as nn
import torch.optim as optim

def train(net, train_loader, epochs=5, learning_rate=0.01, momentum=0.9):
    # Define the loss function
    criterion = nn.CrossEntropyLoss()

    # Define the optimizer
    optimizer = optim.SGD(
        net.parameters(),
        lr=learning_rate,
        momentum=momentum
    )

    # Train the network
    net.train()

    for epoch in range(epochs):
        running_loss = 0.0
        correct = 0
        total = 0

        for images, labels in train_loader:
            # Flatten images from (batch_size, 1, 28, 28) to (batch_size, 784)
            images = images.view(images.size(0), -1)
```

Public - عام

Public - عام


```python
            # Clear previous gradients
            optimizer.zero_grad()

            # Forward pass
            outputs = net(images)

            # Calculate loss
            loss = criterion(outputs, labels)

            # Backward pass
            loss.backward()

            # Update model parameters
            optimizer.step()

            # Track loss and accuracy
            running_loss += loss.item()
            predictions = outputs.argmax(dim=1)
            correct += (predictions == labels).sum().item()
            total += labels.size(0)

        epoch_loss = running_loss / len(train_loader)
        accuracy = 100 * correct / total

        print(
            f"Epoch [{epoch + 1}/{epochs}], "
            f"Loss: {epoch_loss:.4f}, "
            f"Accuracy: {accuracy:.2f}%"
        )

    print("Training completed!")
```

[8] Python

```python
# Train the Neural Network

model = SimpleNet()

train(
    net=model,
    train_loader=train_loader,
    epochs=5,
    learning_rate=0.01,
    momentum=0.9
)
```

[7] Python

```
Epoch [1/5], Loss: 0.3859, Accuracy: 89.04%
Epoch [2/5], Loss: 0.1774, Accuracy: 94.92%
Epoch [3/5], Loss: 0.1258, Accuracy: 96.36%
Epoch [4/5], Loss: 0.0976, Accuracy: 97.10%
Epoch [5/5], Loss: 0.0797, Accuracy: 97.71%
Training completed!
```

```python
# Task 8: Helper functions for tuning (quiet training + evaluation)

import torch
import torch.nn as nn
import torch.optim as optim

def train_quiet(net, train_loader, epochs=5, learning_rate=0.01, momentum=0.9):
    # Same training loop as Task 5/6, without the per-batch printing
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(net.parameters(), lr=learning_rate, momentum=momentum)

    net.train()
    for epoch in range(epochs):
        running_loss, correct, total = 0.0, 0, 0
        for images, labels in train_loader:
            optimizer.zero_grad()
            outputs = net(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
            correct += (outputs.argmax(dim=1) == labels).sum().item()
            total += labels.size(0)

    # Loss and accuracy of the last epoch
    return running_loss / len(train_loader), 100 * correct / total
```

```python
def evaluate(net, test_loader):
    # Same as the Task 7 test function, without printing
    criterion = nn.CrossEntropyLoss()
    net.eval()
    test_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():
        for images, labels in test_loader:
            outputs = net(images)
            test_loss += criterion(outputs, labels).item()
            correct += (outputs.argmax(dim=1) == labels).sum().item()
            total += labels.size(0)
    return test_loss / len(test_loader), 100 * correct / total
```

[13] Python

Public - عام

Public - عام


```python
import pandas as pd

learning_rates = [0.001, 0.01, 0.1]
momentums = [0.0, 0.5, 0.9]
EPOCHS = 5

lr_results = []
for lr in learning_rates:
    for m in momentums:
        torch.manual_seed(42)
        net_lr = SimpleNet()                    # same architecture as Task 4
        train_loss, train_acc = train_quiet(net_lr, train_loader,
                                            epochs=EPOCHS, learning_rate=lr, momentum=m)
        test_loss_lr, test_acc_lr = evaluate(net_lr, test_loader)

        lr_results.append({
            "Learning Rate": lr, "Momentum": m,
            "Train Loss": round(train_loss, 4), "Train Acc (%)": round(train_acc, 2),
            "Test Loss": round(test_loss_lr, 4), "Test Acc (%)": round(test_acc_lr, 2)
        })
        print(f"lr={lr:<6} momentum={m:<4} | Train Loss={train_loss:.4f} | "
              f"Test Loss={test_loss_lr:.4f} | Test Acc={test_acc_lr:.2f}%")

lr_df = pd.DataFrame(lr_results).sort_values(["Test Acc (%)", "Test Loss"],
                                             ascending=[False, True]).reset_index(drop=True)
print("\nResults sorted from best to worst:")
display(lr_df)

best_lr = lr_df.loc[0, "Learning Rate"]
best_momentum = lr_df.loc[0, "Momentum"]
print(f"\nBest parameters: learning rate = {best_lr}, momentum = {best_momentum} "
      f"-> Test Loss = {lr_df.loc[0, 'Test Loss']}, Test Accuracy = {lr_df.loc[0, 'Test Acc (%)']}%")
```

[14] Python

```
lr=0.001  momentum=0.0  | Train Loss=0.7506 | Test Loss=0.6625 | Test Acc=85.52%
lr=0.001  momentum=0.5  | Train Loss=0.4910 | Test Loss=0.4382 | Test Acc=88.83%
lr=0.001  momentum=0.9  | Train Loss=0.2887 | Test Loss=0.2678 | Test Acc=92.42%
lr=0.01   momentum=0.0  | Train Loss=0.2886 | Test Loss=0.2664 | Test Acc=92.50%
lr=0.01   momentum=0.5  | Train Loss=0.2202 | Test Loss=0.2051 | Test Acc=94.17%
lr=0.01   momentum=0.9  | Train Loss=0.0811 | Test Loss=0.0881 | Test Acc=97.44%
lr=0.1    momentum=0.0  | Train Loss=0.0807 | Test Loss=0.0876 | Test Acc=97.32%
lr=0.1    momentum=0.5  | Train Loss=0.0546 | Test Loss=0.0726 | Test Acc=97.76%
lr=0.1    momentum=0.9  | Train Loss=0.1024 | Test Loss=0.1675 | Test Acc=96.02%

Results sorted from best to worst:

...

Best parameters: learning rate = 0.1, momentum = 0.5 -> Test Loss = 0.0726, Test Accuracy = 97.76%
```

```python
# Task 8 - Part 1: Visualize the learning rate / momentum results

import matplotlib.pyplot as plt

acc_table = lr_df.pivot(index="Learning Rate", columns="Momentum", values="Test Acc (%)")
loss_table = lr_df.pivot(index="Learning Rate", columns="Momentum", values="Test Loss")

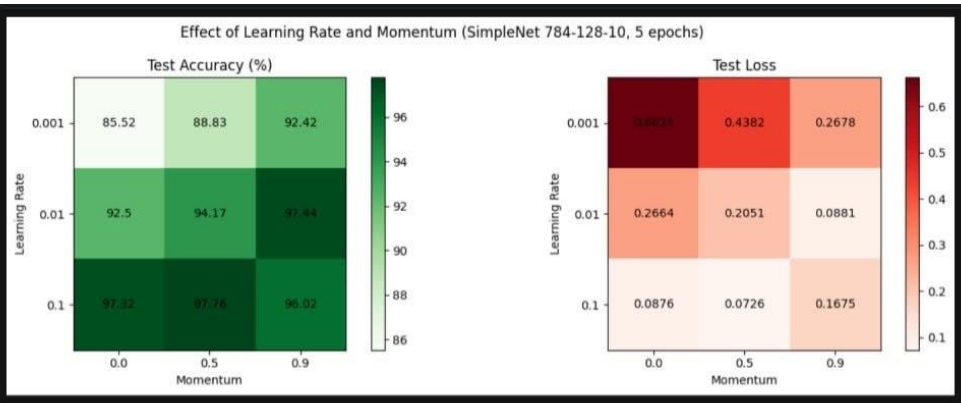
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
for ax, table, title, cmap in [(axes[0], acc_table, "Test Accuracy (%)", "Greens"),
                               (axes[1], loss_table, "Test Loss", "Reds")]:
    im = ax.imshow(table.values, cmap=cmap)
    ax.set_xticks(range(len(table.columns)))
    ax.set_xticklabels(table.columns)
    ax.set_yticks(range(len(table.index)))
    ax.set_yticklabels(table.index)
    ax.set_xlabel("Momentum")
    ax.set_ylabel("Learning Rate")
    ax.set_title(title)
    for i in range(table.shape[0]):
        for j in range(table.shape[1]):
            ax.text(j, i, f"{table.values[i, j]:.4g}", ha="center", va="center")
    fig.colorbar(im, ax=ax)

plt.suptitle("Effect of Learning Rate and Momentum (SimpleNet 784-128-10, 5 epochs)")
plt.tight_layout()
plt.show()
```

[15] Python

Public - عام

Public - عام




```python
# Task 8 - Part 2: Flexible network with any number of hidden layers / neurons

class FlexibleNet(nn.Module):
    def __init__(self, hidden_sizes, input_size=784, num_classes=10):
        super(FlexibleNet, self).__init__()
        layers = []
        in_features = input_size
        for h in hidden_sizes:
            layers.append(nn.Linear(in_features, h))   # fully connected layer
            layers.append(nn.ReLU())                   # non-linearity
            in_features = h
        layers.append(nn.Linear(in_features, num_classes))  # output layer
        self.layers = nn.Sequential(*layers)

    def forward(self, x):
        x = torch.flatten(x, start_dim=1)   # (N, 1, 28, 28) -> (N, 784)
        return self.layers(x)

# Quick check
print(FlexibleNet([256, 128]))
```

[16] Python

```
FlexibleNet(
  (layers): Sequential(
    (0): Linear(in_features=784, out_features=256, bias=True)
    (1): ReLU()
    (2): Linear(in_features=256, out_features=128, bias=True)
    (3): ReLU()
    (4): Linear(in_features=128, out_features=10, bias=True)
  )
)
```

```python
# Task 8 - Part 2: Train and compare different architectures

architectures = [
    [64],           # 1 hidden layer
    [128],          # 1 hidden layer (same as Task 4)
    [256],          # 1 hidden layer
    [512],          # 1 hidden layer
    [128, 64],      # 2 hidden layers
    [256, 128],     # 2 hidden layers
    [512, 256],     # 2 hidden layers
    [256, 128, 64],  # 3 hidden layers
    [512, 256, 128],  # 3 hidden layers
]

arch_results = []
for hidden in architectures:
    torch.manual_seed(42)
    net_arch = FlexibleNet(hidden)
    n_params = sum(p.numel() for p in net_arch.parameters())
    train_loss, train_acc = train_quiet(net_arch, train_loader, epochs=EPOCHS,
                                        learning_rate=best_lr, momentum=best_momentum)
    test_loss_a, test_acc_a = evaluate(net_arch, test_loader)

    name = "784-" + "-".join(map(str, hidden)) + "-10"
    arch_results.append({
        "Architecture": name, "Hidden Layers": len(hidden), "Parameters": n_params,
        "Train Loss": round(train_loss, 4), "Train Acc (%)": round(train_acc, 2),
        "Test Loss": round(test_loss_a, 4), "Test Acc (%)": round(test_acc_a, 2)
    })
    print(f"{name:<20} | layers={len(hidden)} | params={n_params:>7} | "
          f"Test Loss={test_loss_a:.4f} | Test Acc={test_acc_a:.2f}%")
```

[17] Python

Public - عام

Public - عام


```
        })
        print(f"{name:<20} | layers={len(hidden)} | params={n_params:>7} | "
              f"Test Loss={test_loss_a:.4f} | Test Acc={test_acc_a:.2f}%")

    arch_df = pd.DataFrame(arch_results).sort_values(["Test Acc (%)", "Test Loss"],
                                                     ascending=[False, True]).reset_index(drop=True)
    print("\nResults sorted from best to worst:")
    display(arch_df)

    best_arch_name = arch_df.loc[0, "Architecture"]
    best_hidden = [int(h) for h in best_arch_name.split("-")[1:-1]]
    print(f"\nBest architecture: {best_arch_name} -> Test Loss = {arch_df.loc[0, 'Test Loss']}, "
          f"Test Accuracy = {arch_df.loc[0, 'Test Acc (%)']}%")
[17]                                                                                    Python
```

```
784-64-10            | layers=1 | params=  50890 | Test Loss=0.0922 | Test Acc=97.18%
784-128-10           | layers=1 | params= 101770 | Test Loss=0.0726 | Test Acc=97.76%
784-256-10           | layers=1 | params= 203530 | Test Loss=0.0685 | Test Acc=97.79%
784-512-10           | layers=1 | params= 407050 | Test Loss=0.0658 | Test Acc=97.88%
784-128-64-10        | layers=2 | params= 109386 | Test Loss=0.0894 | Test Acc=97.30%
784-256-128-10       | layers=2 | params= 235146 | Test Loss=0.0767 | Test Acc=97.71%
784-512-256-10       | layers=2 | params= 535818 | Test Loss=0.0713 | Test Acc=97.83%
784-256-128-64-10    | layers=3 | params= 242762 | Test Loss=0.0896 | Test Acc=97.46%
784-512-256-128-10   | layers=3 | params= 567434 | Test Loss=0.0717 | Test Acc=97.97%

Results sorted from best to worst:

...

Best architecture: 784-512-256-128-10 -> Test Loss = 0.0717, Test Accuracy = 97.97%
```

```
    # Task 8 - Part 2: Visualize the architecture results

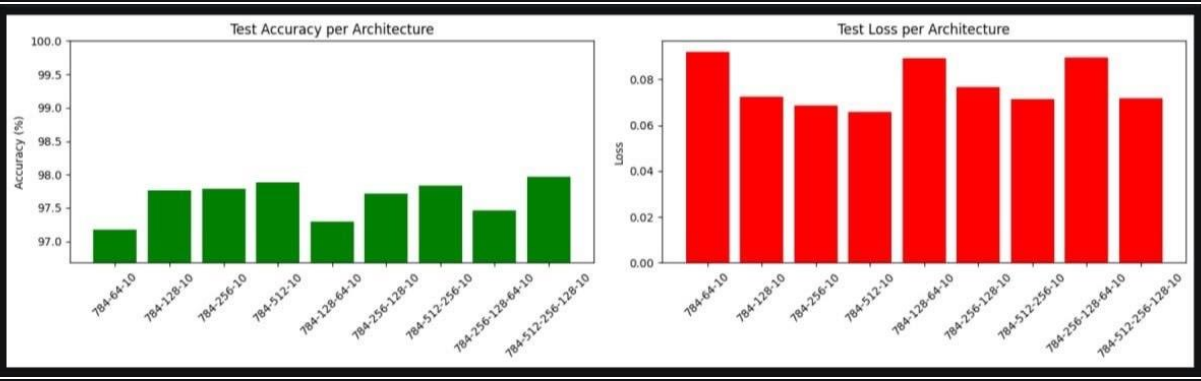
    plot_df = pd.DataFrame(arch_results)   # keep the original order for plotting

    fig, axes = plt.subplots(1, 2, figsize=(15, 4.5))
    axes[0].bar(plot_df["Architecture"], plot_df["Test Acc (%)"], color="green")
    axes[0].set_title("Test Accuracy per Architecture")
    axes[0].set_ylabel("Accuracy (%)")
    axes[0].set_ylim(plot_df["Test Acc (%)"].min() - 0.5, 100)

    axes[1].bar(plot_df["Architecture"], plot_df["Test Loss"], color="red")
    axes[1].set_title("Test Loss per Architecture")
    axes[1].set_ylabel("Loss")

    for ax in axes:
        ax.tick_params(axis="x", rotation=45)

    plt.tight_layout()
    plt.show()
[18]                                                                                    Python
```




```
    # Task 8: Train the final best model

    torch.manual_seed(42)
    best_net = FlexibleNet(best_hidden)
    print(best_net)
    print(f"Learning rate = {best_lr}, Momentum = {best_momentum}, Epochs = {EPOCHS}\n")

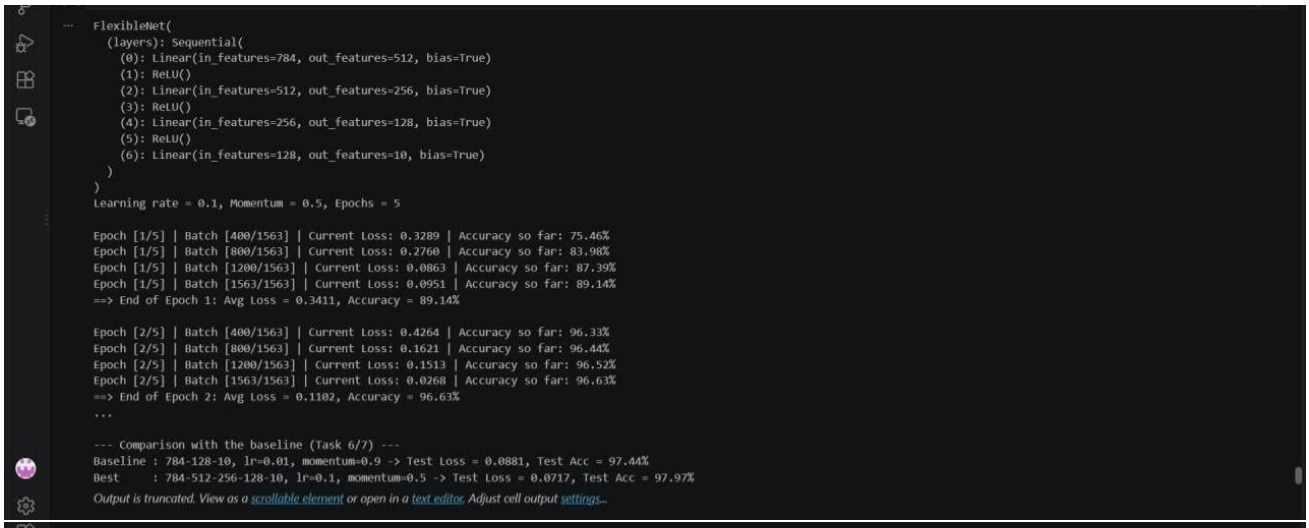
    best_batch_losses, best_epoch_losses, best_epoch_accs = train_with_debug(
        net=best_net,
        train_loader=train_loader,
        epochs=EPOCHS,
        learning_rate=best_lr,
        momentum=best_momentum,
        print_every=400
    )

    print("\n--- Final test results ---")
    best_test_loss, best_test_acc = test(best_net, test_loader)

    print("\n--- Comparison with the baseline (Task 6/7) ---")
    print(f"Baseline : 784-128-10, lr=0.01, momentum=0.9 -> Test Loss = {test_loss:.4f}, Test Acc = {test_accuracy:.2f}%")
    print(f"Best     : {best_arch_name}, lr={best_lr}, momentum={best_momentum} "
          f"-> Test Loss = {best_test_loss:.4f}, Test Acc = {best_test_acc:.2f}%")
[19]                                                                                    Python
```

Public - عام

Public - عام

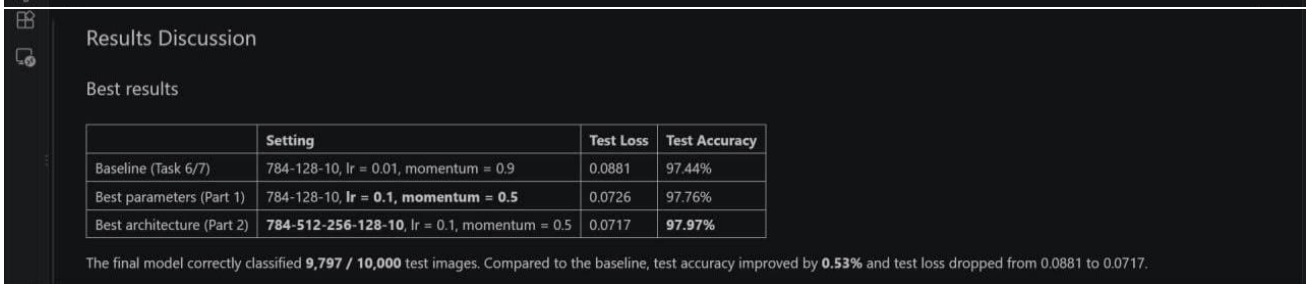


```
FlexibleNet(
  (layers): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=256, bias=True)
    (3): ReLU()
    (4): Linear(in_features=256, out_features=128, bias=True)
    (5): ReLU()
    (6): Linear(in_features=128, out_features=10, bias=True)
  )
)
Learning rate = 0.1, Momentum = 0.5, Epochs = 5

Epoch [1/5] | Batch [400/1563] | Current Loss: 0.3289 | Accuracy so far: 75.46%
Epoch [1/5] | Batch [800/1563] | Current Loss: 0.2760 | Accuracy so far: 83.98%
Epoch [1/5] | Batch [1200/1563] | Current Loss: 0.0863 | Accuracy so far: 87.39%
Epoch [1/5] | Batch [1563/1563] | Current Loss: 0.0951 | Accuracy so far: 89.14%
==> End of Epoch 1: Avg Loss = 0.3411, Accuracy = 89.14%

Epoch [2/5] | Batch [400/1563] | Current Loss: 0.4264 | Accuracy so far: 96.33%
Epoch [2/5] | Batch [800/1563] | Current Loss: 0.1621 | Accuracy so far: 96.44%
Epoch [2/5] | Batch [1200/1563] | Current Loss: 0.1513 | Accuracy so far: 96.52%
Epoch [2/5] | Batch [1563/1563] | Current Loss: 0.0268 | Accuracy so far: 96.63%
==> End of Epoch 2: Avg Loss = 0.1102, Accuracy = 96.63%
...

--- Comparison with the baseline (Task 6/7) ---
Baseline : 784-128-10, lr=0.01, momentum=0.9 -> Test Loss = 0.0881, Test Acc = 97.44%
Best     : 784-512-256-128-10, lr=0.1, momentum=0.5 -> Test Loss = 0.0717, Test Acc = 97.97%
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```



Results Discussion

Best results

	Setting	Test Loss	Test Accuracy
Baseline (Task 6/7)	784-128-10, lr = 0.01, momentum = 0.9	0.0881	97.44%
Best parameters (Part 1)	784-128-10, **lr = 0.1, momentum = 0.5**	0.0726	97.76%
Best architecture (Part 2)	**784-512-256-128-10**, lr = 0.1, momentum = 0.5	0.0717	**97.97%**

The final model correctly classified **9,797 / 10,000** test images. Compared to the baseline, test accuracy improved by **0.53%** and test loss dropped from 0.0881 to 0.0717.

Task 9: Save your source code

Instructions:

- Save your python source code

Deliverable:

- Submit your Github link of your source code along with this document, which includes all screenshots via Blackboard.

- End of Assignment -

Grading Rubric

Criterion	Excellent	Good	Developing	Poor	Marks
Task 1: Install PyTorch	PyTorch installed successfully and PyTorch loaded correctly with clear structure	Installed and PyTorch loaded with minor issues	pyTorch exists but unclear or incomplete	Not completed / missing	**0.5**

Public - عام

Public - عام

Task 2: Download and Load the Dataset	Dataset loaded correctly with clear screenshot showing successful execution	Loaded correctly but screenshot is unclear	Dataset loaded with issues or incomplete evidence	Not loaded / missing screenshot	**0.5**
Task 3: Inspect the Dataset	All clear screenshots	Minor mistakes or missing small details	Several missing description or unclear screenshots	Not answered / incorrect / missing evidence	**0.5**
Task 4: Creating A Simple Network	A simple network correctly craeted, screenshot included	Minor reporting issues	Node used but missing result explanation or screenshot	Not done	**2**
Task 6: Training Function	Training function configured correctly with clear screenshots of output	Minor reporting issues	Training Function used but missing result explanation or screenshot	Not done	**1**
Task 7: Testing Network	Correct Testing network created, screenshot included.	Testing network correct but result description is weak	Testing network exists but unclear results description	Not done	**1**
Task 8: Tuning the parameters and model archirecture	Tuning parameters and model architecture created, screenshot included, and reults description explained accurately	Tuning Parameters and model architecture correct but results description is weak	Tuning Parameters and model architecture corect but incorrect results description	Not done	**1.5**
Task 9: Save your source code	Source code save succefully	Source code save succefully done but minor missing item	Source code save incomplete or file format incorrect	Not submitted / missing	**0.5**
Total					**7.5**

Public - عام